

M.Sc. (Five Year Integrated) in Computer Science (Artificial Intelligence & Data Science)

Seventh Semester

Laboratory Record

21-805-0705: Image and Video Processing Lab

*Submitted in partial fulfillment
of the requirements for the award of degree in
Master of Science (Five Year Integrated)
in Computer Science (Artificial Intelligence & Data Science) of
Cochin University of Science and Technology (CUSAT)
Kochi*



Submitted by

SALMAN FARIS N
(80522018)

DEPARTMENT OF COMPUTER SCIENCE
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)
KOCHI-682022

DECEMBER 2025

DEPARTMENT OF COMPUTER SCIENCE
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)
KOCHI, KERALA-682022



*This is to certify that the software laboratory record for **21-805-0705: Image and Video Processing Lab** is a record of work carried out by **SALMAN FARIS N (80522018)**, in partial fulfillment of the requirements for the award of degree in **Master of Science (Five Year Integrated) in Computer Science (Artificial Intelligence & Data Science)** of Cochin University of Science and Technology (CUSAT), Kochi. The lab record has been approved as it satisfies the academic requirements in respect of the Seventh semester laboratory prescribed for the Master of Science (Five Year Integrated) in Computer Science degree.*

Faculty Member in-charge

Mrs. Shirien K A
Assistant Professor
Department of Computer Science
CUSAT

Dr. Madhu S. Nair
Professor and Head
Department of Computer Science
CUSAT

Table of Content

S.No.	Program Title	Page No.
1	Basic Image Operations	1
2	Histogram Plotting	4
3	Global Histogram Equalization	6
4	Local Histogram Equalization	9
5	Image Enhancement Operations	12
6	Geometric Transformations	19
7	Arithmetic Operations	22
8	Noise Reduction (Averaging)	26
9	Linear Spatial Filters	30
10	Convolution & Correlation	33
11	Median Filtering	36
12	Edge Detection Methods	39
13	RGB Smoothing	42
14	RGB Sharpening	44
15	DFT Filtering	46
16	Freq. Domain Low-pass Filters	49
17	Freq. Domain High-pass Filters	52
18	Line Detection	55
19	Direct Inverse Filtering	58
20	Wiener Filtering	60
21	RGB to HSI Conversion	63
22	HSI Histogram Equalization	66
23	Hough Transform	70
24	Segmentation (Thresholding)	72
25	Segmentation (Region Growing)	74
26	Segmentation (Split & Merge)	75
27	Segmentation (Watershed)	79
28	Optical Flow (Farneback)	82
29	Block Matching (SAD)	85
30	Frame Prediction	89
31	Stereo Disparity & Depth Map	92
32	Obstacle Detection	95

Basic Image Operations

AIM

Write a program to: a) Read an image. b) Get image information. c) Find compression ratio. d) Display negative.

PROGRAM

```
import cv2
from PIL import Image
import os
import numpy as np

# Read the image
image_path = 'input.jpg' # change this to your image path
image = cv2.imread(image_path)

if image is None:
    print("Image not found!")
    exit()

# Get image info
height, width, channels = image.shape
print("Image Information:")
print(f"Dimensions: {width}x{height}")
print(f"Channels: {channels}")
print(f>Data Type: {image.dtype}")

# Save copy with compression and find compression ratio
# Save original as PNG (no compression) for comparison
original_png = 'original.png'
cv2.imwrite(original_png, image)

# Save compressed copy as JPEG
compressed_jpeg = 'compressed.jpg'
cv2.imwrite(compressed_jpeg, image, [cv2.IMWRITE_JPEG_QUALITY, 50])

# Compression Ratio = Uncompressed Size / Compressed Size
uncompressed_size = os.path.getsize(original_png)
compressed_size = os.path.getsize(compressed_jpeg)
compression_ratio = uncompressed_size / compressed_size
```



```
print("\nCompression Info:")
print(f"Uncompressed Size (PNG): {uncompressed_size} bytes")
print(f"Compressed Size (JPEG): {compressed_size} bytes")
print(f"Compression Ratio: {compression_ratio:.2f}")

# Display the negative of the image
negative_image = 255 - image
cv2.imshow("Negative Image", negative_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



```
Image Information:  
Dimensions: 1280x853  
Channels: 3  
Data Type: uint8
```

```
Compression Info:  
Uncompressed Size (PNG): 1105244 bytes  
Compressed Size (JPEG): 61397 bytes  
Compression Ratio: 18.00
```

Histogram Plotting

AIM

Write a program for histogram plotting of an image.

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload Image
print("Please upload your image (e.g., input.jpg):")
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Read the image
image = cv2.imread(image_path)
if image is None:
    print("Error: Could not read the image.")
else:
    print("Image loaded. Displaying it below:")
    cv2_imshow(image)

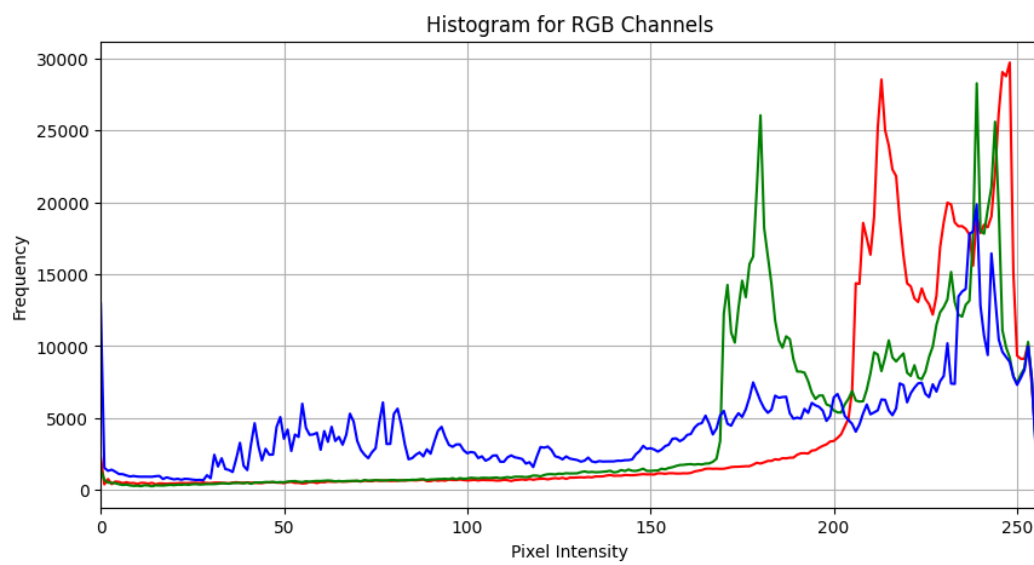
# Convert to RGB for matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Plot histograms
colors = ('r', 'g', 'b') # RGB channels
plt.figure(figsize=(10, 5))
plt.title("Histogram for RGB Channels")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

for i, color in enumerate(colors):
    hist = cv2.calcHist([image], [i], None, [256], [0, 256])
    plt.plot(hist, color=color)
    plt.xlim([0, 256])

plt.grid(True)
plt.show()
```

OUTPUT



Global Histogram Equalization

AIM

Write a program to perform histogram equalization.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload Image
print("Upload an image for histogram equalization:")
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Read and convert to grayscale
image = cv2.imread(image_path)
if image is None:
    print("Error: Cannot read image.")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Histogram Equalization
equalized = cv2.equalizeHist(gray)

# Show Original and Equalized Images
print("\nDisplaying original grayscale and equalized image:")
cv2_imshow(gray)
cv2_imshow(equalized)

# Plot Histograms
def plot_histogram(img, title, color='black'):
    hist = cv2.calcHist([img], [0], None, [256], [0, 256])
    plt.plot(hist, color=color)
    plt.title(title)
```

```
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.xlim([0, 256])
plt.grid(True)

plt.figure(figsize=(12, 5))

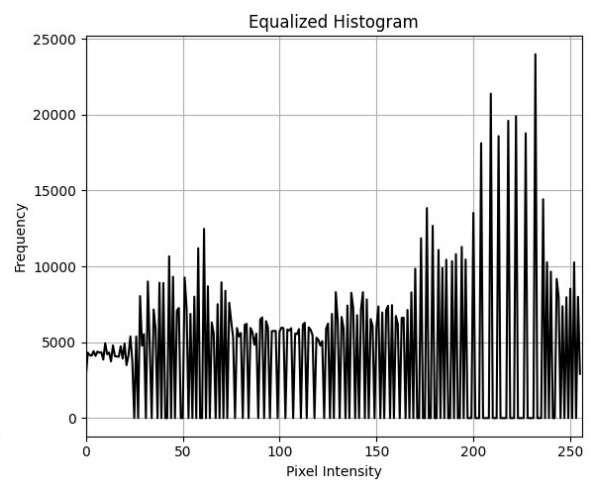
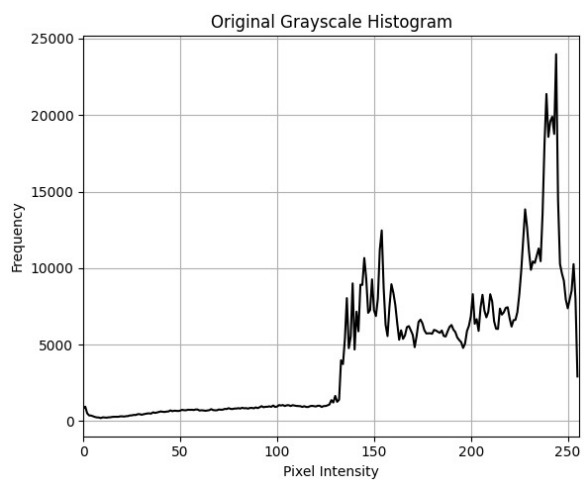
plt.subplot(1, 2, 1)
plot_histogram(gray, 'Original Grayscale Histogram')

plt.subplot(1, 2, 2)
plot_histogram(equalized, 'Equalized Histogram')

plt.tight_layout()
plt.show()
```

OUTPUT





Local Histogram Equalization

AIM

Write a program for performing local histogram equalization.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload Image
print("Upload an image for local histogram equalization:")
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Read and convert to grayscale
image = cv2.imread(image_path)
if image is None:
    print("Error: Could not read the image.")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Local Histogram Equalization
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
local_equalized = clahe.apply(gray)

# Display Original and Local Equalized Images
print("\nDisplaying original grayscale and locally equalized image:")
cv2_imshow(gray)
cv2_imshow(local_equalized)

# Plot Histograms
def plot_histogram(img, title, color='black'):
```



```
hist = cv2.calcHist([img], [0], None, [256], [0, 256])
plt.plot(hist, color=color)
plt.title(title)
plt.xlabel('Pixel Intensity')
plt.ylabel('Frequency')
plt.xlim([0, 256])
plt.grid(True)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plot_histogram(gray, 'Original Grayscale Histogram')

plt.subplot(1, 2, 2)
plot_histogram(local_equalized, 'CLAHE (Local Equalized) Histogram')

plt.tight_layout()
plt.show()
```

OUTPUT



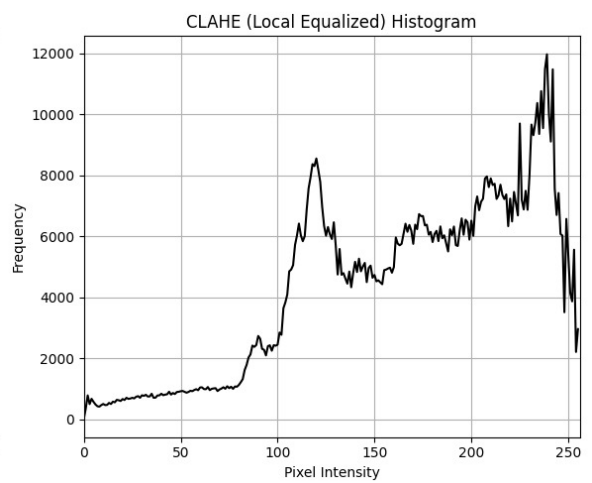
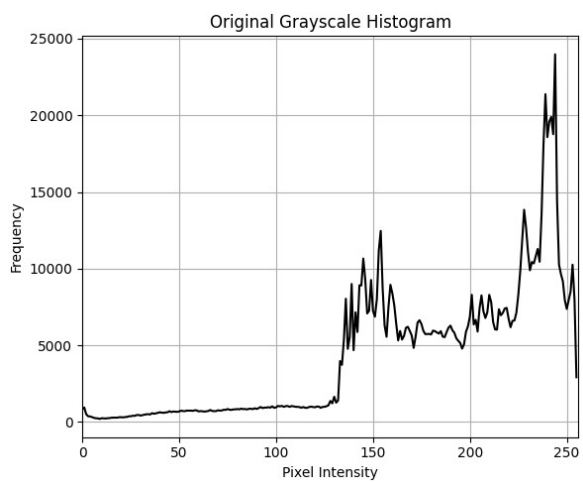


Image Enhancement Operations

AIM

Develop programs for: Brightness, Contrast, Complement, Binary contrast, Slicing, Low-pass, and High-pass filtering.

PROGRAM

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload an image
print("Upload an image:")
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)

# Resize if needed
# img = cv2.resize(img, (512, 512))

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Enhancement operations
# a. Brightness enhancement
bright = cv2.convertScaleAbs(img, alpha=1, beta=50)

# b. Contrast enhancement
contrast = cv2.convertScaleAbs(img, alpha=2, beta=0)

# c. Complement of image
complement = 255 - img

# d. Bi-level contrast
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# e. Brightness slicing
```

```
brightness_sliced = np.where((gray > 100) & (gray < 180), 255,
    ↪ gray).astype(np.uint8)

# Add noise to the image
def add_gaussian_noise(img, mean=0, std=25):
    noise = np.random.normal(mean, std, img.shape).
        astype(np.uint8)
    noisy = cv2.add(img, noise)
    return noisy

# Try salt & pepper noise:
# def add_salt_pepper_noise(img, salt_prob=0.02, pepper_prob=0.02):
#     noisy = img.copy()
#     h, w, c = img.shape
#     num_salt = np.ceil(salt_prob * h * w)
#     num_pepper = np.ceil(pepper_prob * h * w)
#     coords = [np.random.randint(0, i - 1, int(num_salt)) for i in
    ↪ img.shape[:2]]
#     noisy[coords[0], coords[1]] = 255
#     coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in
    ↪ img.shape[:2]]
#     noisy[coords[0], coords[1]] = 0
#     return noisy

img = add_gaussian_noise(img) # Now working with noisy image

# f. Low-pass filtering
low_pass = cv2.GaussianBlur(img, (11, 11), 0)

# g. High-pass filtering
kernel = np.array([[ -1, -1, -1],
                   [ -1,  9, -1],
                   [ -1, -1, -1]])
high_pass = cv2.filter2D(img, -1, kernel)

# Display results
titles = ['Noisy Image', 'Brightness Enhanced', 'Contrast Enhanced',
    ↪ 'Complement',
        'Bi-level Contrast', 'Brightness Sliced', 'Low-pass Filtered',
```

```
        'High-pass Filtered']
images = [img, bright, contrast, complement, binary, brightness_sliced,
↪  low_pass,
        high_pass]

for title, image in zip(titles, images):
    print(f"\n{title}")
    if len(image.shape) == 2:
        cv2_imshow(image)
    else:
        cv2_imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

OUTPUT











Geometric Transformations

AIM

Develop programs for: Translation, Rotation, Scaling, and Skewing.

PROGRAM

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload an image
print("Upload an image:")
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)

rows, cols = img.shape[:2]

# 6a. Translation
tx, ty = 100, 50 # translate x and y
M_translate = np.float32([[1, 0, tx], [0, 1, ty]])
translated = cv2.warpAffine(img, M_translate, (cols, rows))

# 6b. Rotation
angle = 45
M_rotate = cv2.getRotationMatrix2D((cols / 2, rows / 2), angle, 1)
rotated = cv2.warpAffine(img, M_rotate, (cols, rows))

# 6c. Scaling
scaled = cv2.resize(img, None, fx=1.5, fy=1.5,
    ↪ interpolation=cv2.INTER_LINEAR)

# 6d. Skewing (Shearing)
M_skew = np.float32([[1, 0.5, 0], [0.5, 1, 0]])
skewed = cv2.warpAffine(img, M_skew, (int(cols * 1.5), int(rows * 1.5)))

# Display results
titles_geo = ['Translated', 'Rotated', 'Scaled', 'Skewed']
images_geo = [translated, rotated, scaled, skewed]
```

```
for title, image in zip(titles_geo, images_geo):  
    print(f"\n{title}")  
    cv2_imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

OUTPUT

Translation



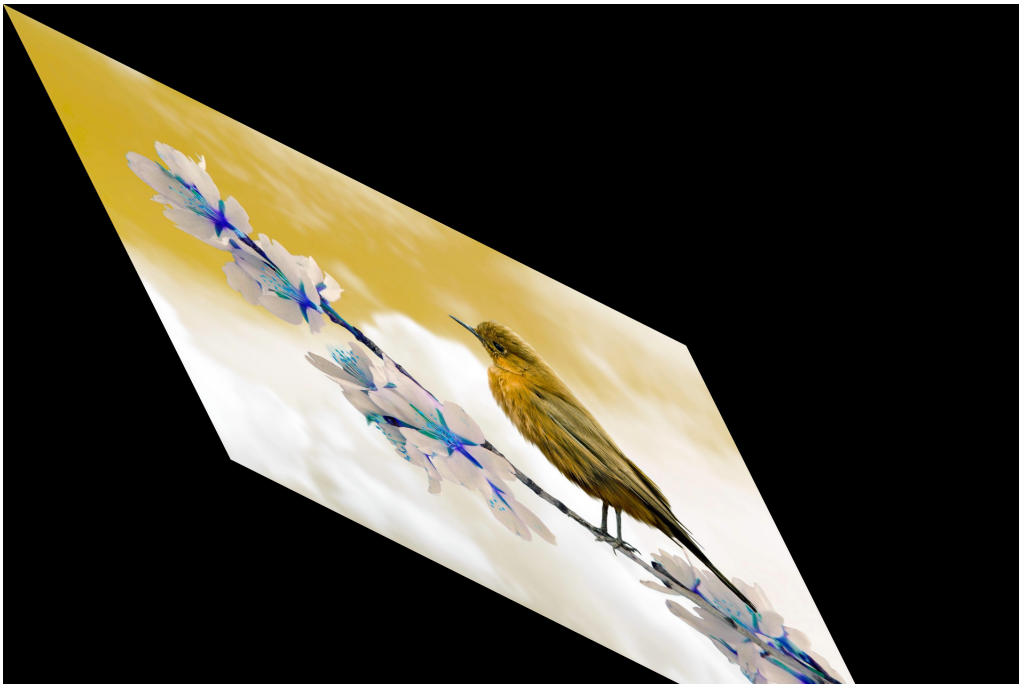
Rotation



Scaling



Skewing



Arithmetic Operations

AIM

Write a program to perform four arithmetic operations between two images.

PROGRAM

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload two images
print("Upload two images:")
uploaded = files.upload()

# Load and resize both images
img1 = cv2.imread(list(uploaded.keys())[0])
img2 = cv2.imread(list(uploaded.keys())[1])
img1 = cv2.resize(img1, (500, 500))
img2 = cv2.resize(img2, (500, 500))

# Basic operations
added = cv2.add(img1, img2)
subtracted = cv2.subtract(img1, img2)

# Multiply and normalize
multiplied = cv2.multiply(img1.astype(np.float32), img2.astype(np.float32))
multiplied = cv2.normalize(multiplied, None, 0, 255, cv2.NORM_MINMAX)
multiplied = multiplied.astype(np.uint8)

# Division with safe handling
img2_safe = img2.astype(np.float32)
img2_safe[img2_safe == 0] = 1
divided = cv2.divide(img1.astype(np.float32), img2_safe)
divided = cv2.normalize(divided, None, 0, 255, cv2.NORM_MINMAX)
divided = divided.astype(np.uint8)

# Show results
print("\nAdded Image:")
cv2_imshow(added)
```

```
print("\nSubtracted Image:")  
cv2_imshow(subtracted)  
  
print("\nMultiplied Image (Normalized):")  
cv2_imshow(multiplied)  
  
print("\nDivided Image (Normalized):")  
cv2_imshow(divided)
```

OUTPUT



Addition



Substraction



Multiplication



Division



Noise Reduction (Averaging)

AIM

Take a noisy image. Reduce noise by averaging the input image by 2, 8, 16, 32, and 128. Compare results.

PROGRAM

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files

# Upload a clean image
print("Upload a clean image:")
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)
img = cv2.resize(img, (400, 400))

# Add Gaussian noise
mean = 0
stddev = 25
gaussian_noise = np.random.normal(mean, stddev,
    ↪ img.shape).astype(np.float32)
noisy_img = img.astype(np.float32) + gaussian_noise
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)

# Convert noisy image to float for averaging
noisy_float = noisy_img.astype(np.float32)

print("\nNoisy Image:")
cv2_imshow(noisy_img)

# Averaging counts
averaging_counts = [2, 8, 16, 32, 128]

# Store results for later PSNR comparison
averaged_results = []

for count in averaging_counts:
```

```
avg_img = np.zeros_like(noisy_float)
for _ in range(count):
    noisy_instance = noisy_float + np.random.normal(0, 10,
    ↪ noisy_float.shape)
    avg_img += noisy_instance
avg_img /= count
avg_img = np.clip(avg_img, 0, 255).astype(np.uint8)
averaged_results.append((count, avg_img)) # <-- store for matplotlib
print(f"\nAveraged over {count} samples:")
cv2_imshow(avg_img)

import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr

# Show images in a grid with PSNR values
plt.figure(figsize=(15, 10))

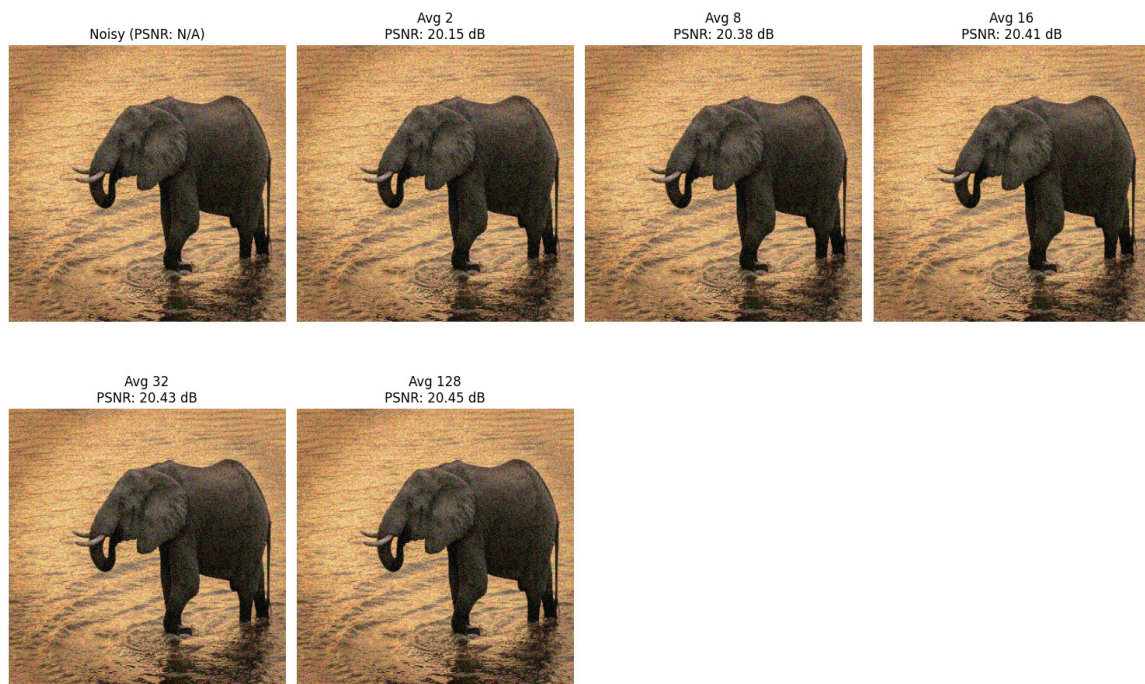
# Add original noisy image for comparison
plt.subplot(2, 4, 1)
plt.imshow(cv2.cvtColor(noisy_img, cv2.COLOR_BGR2RGB))
plt.title("Noisy (PSNR: N/A)")
plt.axis('off')

# Display each averaged image
for idx, (count, img_avg) in enumerate(averaged_results):
    # PSNR compared to the original clean image
    score = psnr(img, img_avg)
    plt.subplot(2, 4, idx + 2)
    plt.imshow(cv2.cvtColor(img_avg, cv2.COLOR_BGR2RGB))
    plt.title(f"Avg {count}\nPSNR: {score:.2f} dB")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT





Linear Spatial Filters

AIM

Write a program which implements all types of linear spatial filters using functions.

PROGRAM

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from google.colab import files

# Load and Prepare Image
def load_image(path, gray=True):
    img = cv2.imread(path)
    if gray:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return img

# Linear Filter Functions
def apply_mean_filter(img, ksize=5):
    return cv2.blur(img, (ksize, ksize))

def apply_gaussian_filter(img, ksize=5, sigma=1):
    return cv2.GaussianBlur(img, (ksize, ksize), sigma)

def apply_laplacian_filter(img):
    return cv2.Laplacian(img, cv2.CV_64F)

def apply_sobel_filter(img, direction='x'):
    if direction == 'x':
        return cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    else:
        return cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

def apply_box_filter(img, ksize=5):
    return cv2.boxFilter(img, -1, (ksize, ksize))

def apply_custom_filter(img, kernel):
    return cv2.filter2D(img, -1, kernel)
```



```
# Display Helper
def show_images(images, titles, cmap='gray'):
    n = len(images)
    plt.figure(figsize=(16, 4))
    for i in range(n):
        plt.subplot(1, n, i+1)
        plt.imshow(images[i], cmap=cmap)
        plt.title(titles[i])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# Main Program
if __name__ == "__main__":
    # Upload image
    print("Please upload an image...")
    uploaded = files.upload()
    img_path = list(uploaded.keys())[0]

    # Load uploaded image
    img = load_image(img_path)

    # Apply filters
    mean_img = apply_mean_filter(img)
    gaussian_img = apply_gaussian_filter(img)
    laplacian_img = apply_laplacian_filter(img)
    sobel_x_img = apply_sobel_filter(img, 'x')
    sobel_y_img = apply_sobel_filter(img, 'y')
    box_img = apply_box_filter(img)

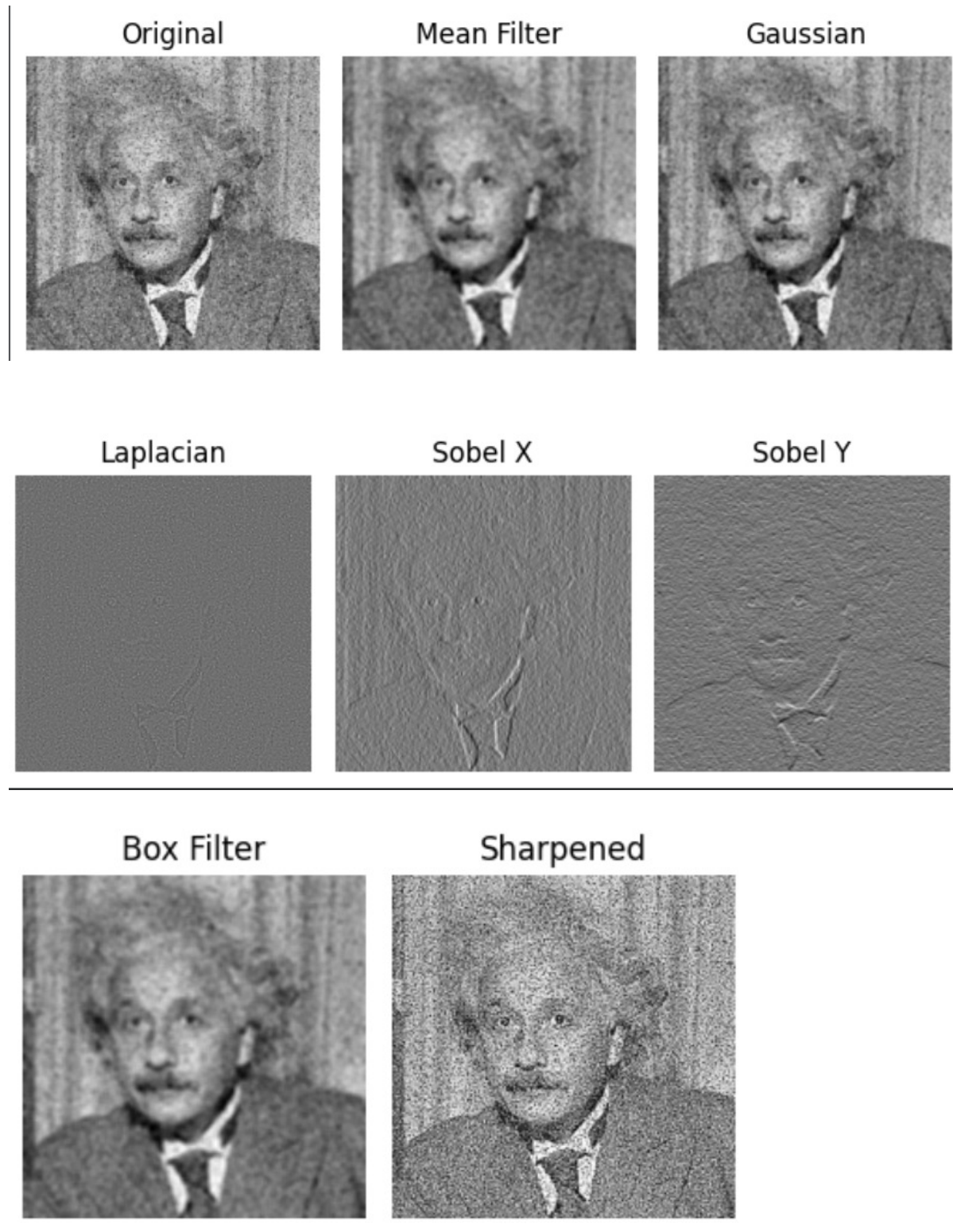
    # Custom kernel (Sharpening)
    sharpen_kernel = np.array([[0, -1, 0],
                                [-1, 5, -1],
                                [0, -1, 0]])
    sharpened_img = apply_custom_filter(img, sharpen_kernel)

    # Display all images
    show_images(
        [img, mean_img, gaussian_img, laplacian_img, sobel_x_img,
         sobel_y_img, box_img, sharpened_img],
        ['Original', 'Mean Filter', 'Gaussian', 'Laplacian', 'Sobel X',
```



```
    'Sobel Y', 'Box Filter', 'Sharpened']  
)
```

OUTPUT



Convolution and Correlation

AIM

Implement image convolution and correlation using a rectangular mask of odd sizes (3x3, 7x7, 11x11) input from text file.

PROGRAM

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from google.colab import files

# Load grayscale image
def load_image():
    print("Upload an image:")
    uploaded = files.upload()
    img_path = list(uploaded.keys())[0]
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    return img

# Load kernel from text file
def load_kernel():
    print("Upload a kernel (ASCII text file):")
    uploaded = files.upload()
    kernel_path = list(uploaded.keys())[0]
    with open(kernel_path, 'r') as f:
        lines = f.readlines()
    kernel = [list(map(float, line.strip().split())) for line in lines]
    return np.array(kernel, dtype=np.float32)

# Convolution
def apply_convolution(img, kernel):
    return cv2.filter2D(img, -1, kernel)

# Correlation (via flipping kernel)
def apply_correlation(img, kernel):
    flipped_kernel = cv2.flip(kernel, -1) # flip both axes
    return cv2.filter2D(img, -1, flipped_kernel)

# Show Images
```

```
def show_images(img_list, title_list):
    plt.figure(figsize=(12, 4))
    for i in range(len(img_list)):
        plt.subplot(1, len(img_list), i+1)
        plt.imshow(img_list[i], cmap='gray')
        plt.title(title_list[i])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# Main
img = load_image()
kernel = load_kernel()

convolved_img = apply_convolution(img, kernel)
correlated_img = apply_correlation(img, kernel)

# Show results
show_images([img, convolved_img, correlated_img],
            ['Original Image', 'Convolution Result', 'Correlation Result'])
```

OUTPUT

3 × 3 kernel



7×7 kernel

Original Image



Convolution Result



Correlation Result



11×11 kernel

Original Image



Convolution Result



Correlation Result



Median Filtering

AIM

Write a program for implementing median filtering on an image with salt and pepper noise.

PROGRAM

```
# Import Libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.util import random_noise

# Load the image
# Make sure to have an image in the same folder or give full path
image = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    raise FileNotFoundError("Image not found! Please check the path.")

plt.figure(figsize=(5,5))
plt.title("Original Image")
plt.imshow(image, cmap="gray")
plt.axis("off")
plt.show()

# Add Salt and Pepper Noise
noisy_img = random_noise(image, mode='s&p', amount=0.05) # 5% noise
noisy_img = (noisy_img * 255).astype(np.uint8)

plt.figure(figsize=(5,5))
plt.title("Image with Salt and Pepper Noise")
plt.imshow(noisy_img, cmap="gray")
plt.axis("off")
plt.show()

# Apply Median Filtering
median_filtered = cv2.medianBlur(noisy_img, 3)

plt.figure(figsize=(5,5))
plt.title("Median Filtered Image")
```

```
plt.imshow(median_filtered, cmap="gray")
plt.axis("off")
plt.show()

# Compare Results
fig, axs = plt.subplots(1, 3, figsize=(15, 5))
titles = ["Original", "Noisy (Salt & Pepper)", "Median Filtered"]
images = [image, noisy_img, median_filtered]

for i in range(3):
    axs[i].imshow(images[i], cmap='gray')
    axs[i].set_title(titles[i])
    axs[i].axis('off')

plt.show()
```

OUTPUT

Original Image



Image with Salt and Pepper Noise



Median Filtered Image



Edge Detection Methods

AIM

Detect edges using Sobel, Prewitt, Roberts, LoG, and Canny methods. Compare performance.

PROGRAM

```
# Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import filters

# Function to display images side by side
def show_images(images, titles, cmap="gray"):
    plt.figure(figsize=(15, 8))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(2, 3, i + 1)
        plt.imshow(img, cmap=cmap)
        plt.title(title)
        plt.axis("off")
    plt.show()

# Load the image
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError("Image not found! Please put 'sample.jpg' in
        the working directory.")

plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

# Sobel Edge Detection
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = np.hypot(sobelx, sobely).astype(np.uint8)
```

```
# Prewitt Edge Detection (using convolution kernels)
kernelx = np.array([[ -1, 0, 1],
                    [ -1, 0, 1],
                    [ -1, 0, 1]])
kernely = np.array([[ -1, -1, -1],
                    [  0,  0,  0],
                    [  1,  1,  1]])
prewittx = cv2.filter2D(img, -1, kernelx)
prewitty = cv2.filter2D(img, -1, kernely)
prewitt = cv2.add(prewittx, prewitty)

# Cell 5: Roberts Edge Detection
roberts_cross_v = np.array([[1, 0],
                           [0, -1]])
roberts_cross_h = np.array([[0, 1],
                           [-1, 0]])
roberts_v = cv2.filter2D(img, -1, roberts_cross_v)
roberts_h = cv2.filter2D(img, -1, roberts_cross_h)
roberts = cv2.add(roberts_v, roberts_h)

# Cell 6: Laplacian of Gaussian (LoG)
blurred = cv2.GaussianBlur(img, (3, 3), 0)
log = cv2.Laplacian(blurred, cv2.CV_64F)
log = np.uint8(np.absolute(log))

# Cell 7: Canny Edge Detection
canny = cv2.Canny(img, 100, 200)

# Cell 8: Show all results together
show_images(
    [img, sobel, prewitt, roberts, log, canny],
    ["Original", "Sobel", "Prewitt", "Roberts", "LoG", "Canny"]
)
```

OUTPUT

Original



Sobel



Prewitt



Roberts



LoG



Canny



RGB Smoothing

AIM

Write a program for smoothing an RGB color image with a linear spatial filter.

PROGRAM

```
# Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Function to display images
def show_images(images, titles):
    plt.figure(figsize=(12, 6))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(1, len(images), i + 1)
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title(title)
        plt.axis("off")
    plt.tight_layout()
    plt.show()

# Load an RGB image
img = cv2.imread("sample.jpg")    # change filename if needed
if img is None:
    raise FileNotFoundError("Image not found! Please put 'sample.jpg' in
        the working directory.")

show_images([img], ["Original Image"])

# Apply linear spatial filters (smoothing)
# Mean filter (Averaging filter)
smoothed_3x3 = cv2.blur(img, (3, 3))
smoothed_5x5 = cv2.blur(img, (5, 5))
smoothed_11x11 = cv2.blur(img, (11, 11))

# Display results
show_images(
    [img, smoothed_3x3, smoothed_5x5, smoothed_11x11],
    ["Original", "Smoothed (3x3)", "Smoothed (5x5)", "Smoothed (11x11)"]
)
```

OUTPUT

Original



Smoothed (3x3)



Smoothed (5x5)



Smoothed (11x11)



RGB Sharpening

AIM

Write a program for sharpening an RGB color image with the Laplacian filter mask.

PROGRAM

```
# Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Function to display images
def show_images(images, titles):
    plt.figure(figsize=(12, 6))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(1, len(images), i + 1)
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title(title)
        plt.axis("off")
    plt.tight_layout()
    plt.show()

# Load image
img = cv2.imread("sample.jpg") # replace with your image filename
if img is None:
    raise FileNotFoundError("Image not found! Please put 'sample.jpg' in
        the working directory.")

show_images([img], ["Original RGB Image"])

# Apply Laplacian filter
laplacian = cv2.Laplacian(img, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# Sharpen the image (Original - Laplacian)
sharpened = cv2.addWeighted(img, 1.0, laplacian, -1.0, 0)

# Display results
show_images([img, laplacian, sharpened],
            ["Original", "Laplacian (Edges)", "Sharpened with Laplacian"])
```

OUTPUT



Laplacian (Edges)



Sharpened with Laplacian



DFT Filtering

AIM

Implement DFT filtering with (a) padding and (b) without padding. Compare results.

PROGRAM

```
# Imports
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show_images(images, titles, cmap="gray"):
    n = len(images)
    plt.figure(figsize=(15, 6))
    for i in range(n):
        plt.subplot(1, n, i+1)
        plt.imshow(images[i], cmap=cmap)
        plt.title(titles[i])
        plt.axis("off")
    plt.tight_layout()
    plt.show()

# Load and preprocess image
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE) # Load in grayscale
if img is None:
    raise FileNotFoundError("Image not found! Please place 'sample.jpg' in
    working directory.")

# Resize to 256x256 as per question
img = cv2.resize(img, (256, 256))

show_images([img], ["Original 256x256 Image"])

# DFT without padding
dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft) # Shift zero freq to center
magnitude_spectrum_no_pad = 20 * np.log(cv2.magnitude(dft_shift[:, :, 0],
dft_shift[:, :, 1]) + 1)

# DFT with padding
rows, cols = img.shape
```

```
nrows = cv2.getOptimalDFTSize(rows)
ncols = cv2.getOptimalDFTSize(cols)

# Pad image
right = ncols - cols
bottom = nrows - rows
padded = cv2.copyMakeBorder(img, 0, bottom, 0, right, cv2.BORDER_CONSTANT,
value=0)

dft_pad = cv2.dft(np.float32(padded), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift_pad = np.fft.fftshift(dft_pad)
magnitude_spectrum_pad = 20 * np.log(cv2.magnitude(dft_shift_pad[:, :, 0],
dft_shift_pad[:, :, 1]) + 1)

# Compare results
show_images(
    [img, magnitude_spectrum_no_pad, padded, magnitude_spectrum_pad],
    ["Original (256x256)", "DFT (No Padding)", "Padded Image", "DFT (With
    ↪ Padding)"]
)
```

OUTPUT

Original (256x256)



DFT Low-pass (No Padding)



DFT Low-pass (With Padding)



Frequency Domain Low-pass Filters

AIM

Implement various low-pass or smoothing frequency domain filters.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Frequency Domain Filter Functions
def ideal_low_pass(shape, cutoff):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            if np.sqrt((u-crow)**2 + (v-ccol)**2) <= cutoff:
                mask[u, v] = 1
    return mask

def butterworth_low_pass(shape, cutoff, order=2):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            D = np.sqrt((u-crow)**2 + (v-ccol)**2)
            mask[u, v] = 1 / (1 + (D/cutoff)**(2*order))
    return mask

def gaussian_low_pass(shape, cutoff):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            D2 = (u-crow)**2 + (v-ccol)**2
            mask[u, v] = np.exp(-D2 / (2*(cutoff**2)))
    return mask
```

```
# Apply Filter
def apply_filter(img, mask):
    # DFT
    dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)

    # Apply mask
    dft_shift[:, :, 0] *= mask
    dft_shift[:, :, 1] *= mask

    # Inverse DFT
    f_ishift = np.fft.ifftshift(dft_shift)
    img_back = cv2.idft(f_ishift)
    img_back = cv2.magnitude(img_back[:, :, 0], img_back[:, :, 1])

    return img_back

# Main Program
# Load and resize image
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

cutoff = 30 # cutoff frequency

# Generate filters
ilpf = ideal_low_pass(img.shape, cutoff)
blpf = butterworth_low_pass(img.shape, cutoff, order=2)
glpf = gaussian_low_pass(img.shape, cutoff)

# Apply filters
ilpf_img = apply_filter(img, ilpf)
blpf_img = apply_filter(img, blpf)
glpf_img = apply_filter(img, glpf)

# Display Results
titles = ["Original", "Ideal LPF", "Butterworth LPF", "Gaussian LPF"]
images = [img, ilpf_img, blpf_img, glpf_img]

plt.figure(figsize=(12,6))
for i in range(4):
```



```
plt.subplot(2,2,i+1)
plt.imshow(images[i], cmap='gray')
plt.title(titles[i])
plt.axis('off')
plt.tight_layout()
plt.show()
```

OUTPUT

Original



Ideal LPF



Butterworth LPF



Gaussian LPF



Frequency Domain High-pass Filters

AIM

Implement various high-pass or sharpening frequency domain filters.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Frequency Domain High-Pass Filters
def ideal_high_pass(shape, cutoff):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.ones((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            if np.sqrt((u-crow)**2 + (v-ccol)**2) <= cutoff:
                mask[u, v] = 0
    return mask

def butterworth_high_pass(shape, cutoff, order=2):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            D = np.sqrt((u-crow)**2 + (v-ccol)**2)
            if D == 0:
                mask[u,v] = 0
            else:
                mask[u, v] = 1 / (1 + (cutoff/D)**(2*order))
    return mask

def gaussian_high_pass(shape, cutoff):
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
```

```
D2 = (u-crow)**2 + (v-ccol)**2
mask[u, v] = 1 - np.exp(-D2 / (2*(cutoff**2)))
return mask

# Apply Filter
def apply_filter(img, mask):
    # DFT
    dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)

    # Apply mask
    dft_shift[:, :, 0] *= mask
    dft_shift[:, :, 1] *= mask

    # Inverse DFT
    f_ishift = np.fft.ifftshift(dft_shift)
    img_back = cv2.idft(f_ishift)
    img_back = cv2.magnitude(img_back[:, :, 0], img_back[:, :, 1])

    return img_back

# Main Program
# Load and resize image
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

cutoff = 30 # cutoff frequency

# Generate filters
ihpf = ideal_high_pass(img.shape, cutoff)
bhpfp = butterworth_high_pass(img.shape, cutoff, order=2)
ghpfp = gaussian_high_pass(img.shape, cutoff)

# Apply filters
ihpf_img = apply_filter(img, ihpf)
bhpfp_img = apply_filter(img, bhpfp)
ghpfp_img = apply_filter(img, ghpfp)

# Display Results
titles = ["Original", "Ideal HPF", "Butterworth HPF", "Gaussian HPF"]
images = [img, ihpf_img, bhpfp_img, ghpfp_img]
```

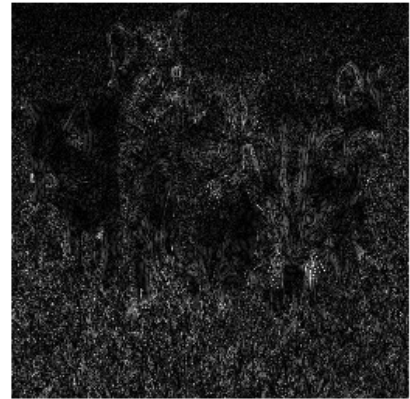
```
plt.figure(figsize=(12,6))
for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```

OUTPUT

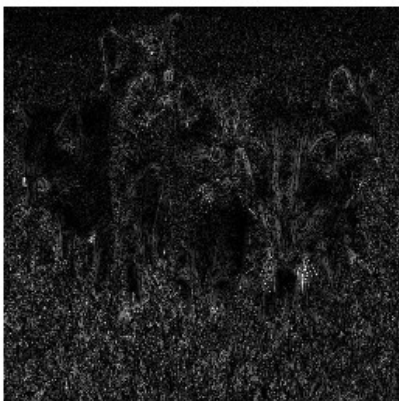
Original



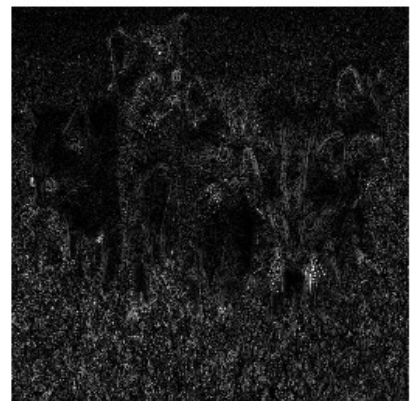
Ideal HPF



Butterworth HPF



Gaussian HPF



Line Detection

AIM

Write a program to detect straight lines using horizontal, vertical, and diagonal filter masks.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load Image (Grayscale)
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Define Line Detection Masks

# Horizontal mask
horizontal_mask = np.array([[ -1, -1, -1],
                             [ 2,  2,  2],
                             [-1, -1, -1]])

# Vertical mask
vertical_mask = np.array([[ -1,  2, -1],
                           [-1,  2, -1],
                           [-1,  2, -1]])

# Diagonal 45° mask
diag1_mask = np.array([[ 2, -1, -1],
                        [-1,  2, -1],
                        [-1, -1,  2]])

# Diagonal 135° mask
diag2_mask = np.array([[ -1, -1,  2],
                        [-1,  2, -1],
                        [ 2, -1, -1]])

# Apply Filters
horizontal_lines = cv2.filter2D(img, -1, horizontal_mask)
vertical_lines   = cv2.filter2D(img, -1, vertical_mask)
diag1_lines      = cv2.filter2D(img, -1, diag1_mask)
```

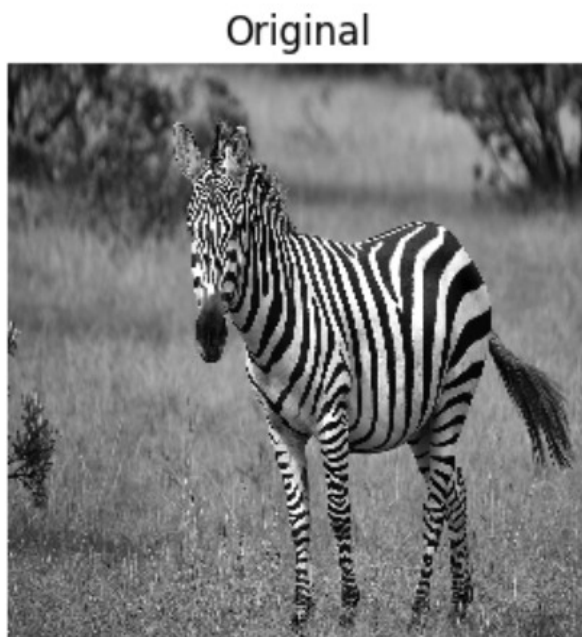
```
diag2_lines      = cv2.filter2D(img, -1, diag2_mask)

# Display Results
titles = ["Original", "Horizontal Lines", "Vertical Lines", "Diagonal 45",
↪  "Diagonal 135"]
images = [img, horizontal_lines, vertical_lines, diag1_lines, diag2_lines]

plt.figure(figsize=(12,6))
for i in range(5):
    plt.subplot(2,3,i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



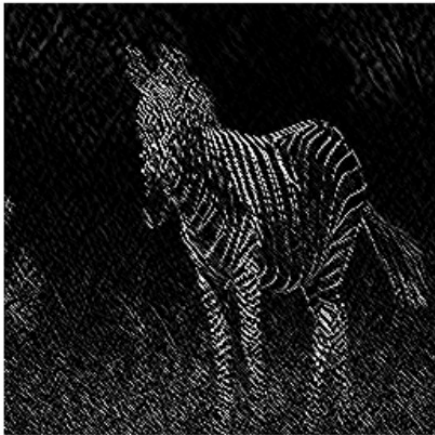
Horizontal Lines



Vertical Lines



Diagonal 45°



Diagonal 135°



Direct Inverse Filtering

AIM

Write a program which restores a degraded image using direct inverse filtering.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Add motion blur
def motion_blur(image, size=15):
    kernel = np.zeros((size, size))
    kernel[int((size - 1)/2), :] = np.ones(size)
    kernel /= size
    return cv2.filter2D(image, -1, kernel), kernel

# Inverse Filtering Function
def inverse_filtering(degraded, H):
    # DFT of degraded image
    G = np.fft.fft2(degraded)
    H_f = np.fft.fft2(H, s=degraded.shape)

    # Avoid divide by zero (add epsilon)
    epsilon = 1e-6
    F_hat = G / (H_f + epsilon)

    # Inverse DFT to get restored image
    f_restored = np.fft.ifft2(F_hat)
    f_restored = np.abs(f_restored)
    return np.uint8(np.clip(f_restored, 0, 255))

# Main Program
# Load grayscale image
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Apply motion blur (degradation)
degraded, kernel = motion_blur(img, size=15)

# Restore using inverse filtering
```

```
restored = inverse_filtering(degraded, kernel)

# Display Results
titles = ["Original", "Degraded (Motion Blur)", "Restored (Inverse Filter)"]
images = [img, degraded, restored]

plt.figure(figsize=(12, 4))
for i in range(3):
    plt.subplot(1, 3, i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



Wiener Filtering

AIM

Implement Wiener filtering for linear image restoration using (a) constant ratio and (b) auto-correlation function.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load Image
img = cv2.imread("image.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found! Place 'image.png' in the same folder.")
img = cv2.resize(img, (256, 256))

# Create Motion Blur PSF (Degradation Function)
def motion_blur_psf(shape, a=0.1, b=0.1, T=1):
    M, N = shape
    U, V = np.meshgrid(np.arange(-N//2, N//2), np.arange(-M//2, M//2))
    H = T * (np.sinc(a*U + b*V) + 1e-8) * np.exp(-1j*np.pi*(a*U + b*V))
    return np.fft.ifftshift(H)

H = motion_blur_psf(img.shape)

# Fourier transform of original image
F = np.fft.fft2(img)

# Apply degradation
G = H * F

# Add Gaussian noise
noise = np.random.normal(0, 10, img.shape)
G_noisy = G + np.fft.fft2(noise)

# Degraded image
g = np.abs(np.fft.ifft2(G_noisy))

# A: Wiener Filter (Constant K)
```

```
K = 0.01 # constant ratio
H_conj = np.conj(H)
W_filter_const = (H_conj / (np.abs(H)**2 + K))
F_hat_const = W_filter_const * G_noisy
f_hat_const = np.abs(np.fft.ifft2(F_hat_const))

# B: Wiener Filter (Autocorrelation Estimate)
# Power spectrum of degraded image
S_gg = np.abs(G_noisy)**2
# Noise power spectrum (estimated from added noise)
S_nn = np.abs(np.fft.fft2(noise))**2

# Wiener filter using auto-correlation
W_filter_auto = (H_conj / (np.abs(H)**2 + (S_nn / (S_gg + 1e-8)))))
F_hat_auto = W_filter_auto * G_noisy
f_hat_auto = np.abs(np.fft.ifft2(F_hat_auto))

# Display Results
plt.figure(figsize=(15, 6))

plt.subplot(1, 4, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(g, cmap='gray')
plt.title("Degraded (Blur + Noise)")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(f_hat_const, cmap='gray')
plt.title("Restored (Wiener, K=const)")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(f_hat_auto, cmap='gray')
plt.title("Restored (Wiener, Auto)")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

OUTPUT



RGB to HSI Conversion

AIM

Write a program to convert an RGB color space to HSI. Display Hue, Saturation, and Intensity images.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def rgb_to_hsi(img):
    # Normalize RGB values to [0,1]
    img = img.astype(np.float32) / 255.0
    R, G, B = img[:, :, 2], img[:, :, 1], img[:, :, 0]

    # Intensity
    I = (R + G + B) / 3.0

    # Saturation
    min_val = np.minimum(np.minimum(R, G), B)
    S = 1 - (3 / (R + G + B + 1e-8)) * min_val # avoid division by zero

    # Hue calculation
    num = 0.5 * ((R - G) + (R - B))
    den = np.sqrt((R - G) ** 2 + (R - B) * (G - B)) + 1e-8
    theta = np.arccos(num / den)

    H = np.zeros_like(R)
    H[B <= G] = theta[B <= G]
    H[B > G] = (2 * np.pi - theta[B > G])

    H = H / (2 * np.pi) # Normalize to [0,1]

    return H, S, I

# Load Image
img = cv2.imread("image.png") # Change to your image path
if img is None:
    raise FileNotFoundError("Image not found!")
img = cv2.resize(img, (256, 256)) # Resize for display
```

```
# Convert to HSI
H, S, I = rgb_to_hsi(img)

# Display
plt.figure(figsize=(12, 4))

plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original RGB")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(H, cmap="hsv")
plt.title("Hue")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(S, cmap="gray")
plt.title("Saturation")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(I, cmap="gray")
plt.title("Intensity")
plt.axis("off")

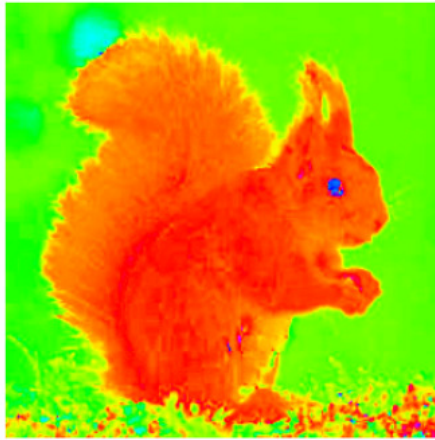
plt.tight_layout()
plt.show()
```

OUTPUT

Original RGB



Hue



Saturation



Intensity



HSI Histogram Equalization

AIM

Histogram equalize the Intensity component of a color image and get a new HSI image. Convert back to RGB.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Function: RGB → HSI =
def rgb_to_hsi(img):
    img = img.astype(np.float32) / 255.0
    B, G, R = cv2.split(img)

    # Intensity
    I = (R + G + B) / 3.0

    # Saturation
    min_val = np.minimum(np.minimum(R, G), B)
    S = 1 - (3 / (R + G + B + 1e-8)) * min_val

    # Hue
    num = 0.5 * ((R - G) + (R - B))
    den = np.sqrt((R - G)**2 + (R - B)*(G - B)) + 1e-8
    theta = np.arccos(num / den)

    H = np.zeros_like(R)
    H[B <= G] = theta[B <= G]
    H[B > G] = (2*np.pi - theta[B > G])
    H = H / (2*np.pi) # Normalize to [0,1]

    return H, S, I

# Function: HSI → RGB
def hsi_to_rgb(H, S, I):
    H = H * 2 * np.pi
    R, G, B = np.zeros_like(H), np.zeros_like(H), np.zeros_like(H)
```

```
# Sector 1 (0 ≤ H < 2π/3)
idx = (H >= 0) & (H < 2*np.pi/3)
B[idx] = I[idx] * (1 - S[idx])
R[idx] = I[idx] * (1 + (S[idx] * np.cos(H[idx]))) / (np.cos(np.pi/3 -
    H[idx]) + 1e-8))
G[idx] = 3*I[idx] - (R[idx] + B[idx])

# Sector 2 (2π/3 ≤ H < 4π/3)
idx = (H >= 2*np.pi/3) & (H < 4*np.pi/3)
H2 = H[idx] - 2*np.pi/3
R[idx] = I[idx] * (1 - S[idx])
G[idx] = I[idx] * (1 + (S[idx] * np.cos(H2))) / (np.cos(np.pi/3 - H2) +
    ↪ 1e-8))
B[idx] = 3*I[idx] - (R[idx] + G[idx])

# Sector 3 (4π/3 ≤ H < 2π)
idx = (H >= 4*np.pi/3) & (H <= 2*np.pi)
H3 = H[idx] - 4*np.pi/3
G[idx] = I[idx] * (1 - S[idx])
B[idx] = I[idx] * (1 + (S[idx] * np.cos(H3))) / (np.cos(np.pi/3 - H3) +
    ↪ 1e-8))
R[idx] = 3*I[idx] - (G[idx] + B[idx])

rgb = np.dstack((R, G, B))
rgb = np.clip(rgb, 0, 1)
return (rgb * 255).astype(np.uint8)

# Load and Resize Image
img = cv2.imread("image.png")
if img is None:
    raise FileNotFoundError("Image not found!")
img = cv2.resize(img, (256, 256))

# Convert to HSI
H, S, I = rgb_to_hsi(img)

# Histogram Equalization on Intensity
I_eq = cv2.equalizeHist((I * 255).astype(np.uint8)) / 255.0

# Reconstruct new HSI → RGB
```

```
rgb_restored = hsi_to_rgb(H, S, I_eq)

# Display Results
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original RGB")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(I, cmap="gray")
plt.title("Original Intensity")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(I_eq, cmap="gray")
plt.title("Equalized Intensity")
plt.axis("off")

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Before Equalization")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(rgb_restored, cv2.COLOR_BGR2RGB))
plt.title("After Equalization (HSI→RGB)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT

Original RGB



Original Intensity



Equalized Intensity



Before Equalization



After Equalization (HSI→RGB)



Hough Transform

AIM

Write a program to detect line segments in a binary image using Hough Transform.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the binary image
img = cv2.imread('binary_image.png', cv2.IMREAD_GRAYSCALE)

# If not already binary, threshold it
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Edge detection (optional for noisy images)
edges = cv2.Canny(binary, 50, 150, apertureSize=3)

# Detect lines using Probabilistic Hough Transform ===
    edges,
    rho=1,
    theta=np.pi/180,
    threshold=50,
    minLineLength=30,
    maxLineGap=10
)

# Draw the detected lines
line_img = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(line_img, (x1, y1), (x2, y2), (0, 0, 255), 2)

# Display results
plt.figure(figsize=(12,5))
plt.subplot(1,3,1)
plt.imshow(binary, cmap='gray')
plt.title('Original Binary Image')
```

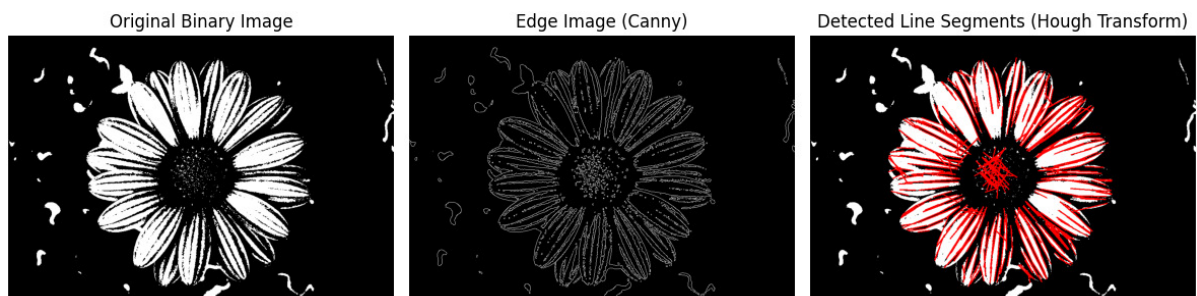
```
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(edges, cmap='gray')
plt.title('Edge Image (Canny)')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB))
plt.title('Detected Line Segments (Hough Transform)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT



Segmentation (Thresholding)

AIM

Write a program to segment non-overlapping blobs based on thresholding.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read and preprocess
img = cv2.imread("image.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add synthetic blobs
output = gray.copy()
h, w = gray.shape
for _ in range(10):
    x, y = np.random.randint(20, w-20), np.random.randint(20, h-20)
    cv2.circle(output, (x, y), 10, 255, -1)

# Threshold
_, thresh = cv2.threshold(output, 128, 255, cv2.THRESH_BINARY)

# Find connected components with stats
num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(thresh)

# Filter out small regions (noise)
min_area = 100 # only keep blobs larger than this
blob_count = np.sum(stats[1:, cv2.CC_STAT_AREA] > min_area)

# Display
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),
plt.title("Original RGB Image"), plt.axis("off")
plt.subplot(1, 3, 2), plt.imshow(output, cmap="gray"), plt.title("Image
with Synthetic Blobs"), plt.axis("off")
plt.subplot(1, 3, 3), plt.imshow(thresh, cmap="gray"),
plt.title(f"Segmented Blobs - {blob_count} large blobs"), plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



Segmentation (Region Growing)

AIM

Write a program to segment blobs based on region growing.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from collections import deque

def region_growing(img, seed, threshold=10):
    h, w = img.shape
    segmented = np.zeros_like(img, np.uint8)
    queue = deque([seed])
    seed_intensity = img[seed]
    while queue:
        x, y = queue.popleft()
        if segmented[x, y] == 0:
            diff = abs(int(img[x, y]) - int(seed_intensity))
            if diff < threshold:
                segmented[x, y] = 255
                for i in range(-1, 2):
                    for j in range(-1, 2):
                        nx, ny = x + i, y + j
                        if 0 <= nx < h and 0 <= ny < w:
                            if segmented[nx, ny] == 0:
                                queue.append((nx, ny))
    return segmented

# Load and preprocess
img = cv2.imread("sample.jpg", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Add blobs
for _ in range(5):
    x, y = np.random.randint(30, 226), np.random.randint(30, 226)
    cv2.circle(img, (x, y), 10, 200 + np.random.randint(-30, 30), -1)

# Find candidate seeds using thresholding
_, binary = cv2.threshold(img, 180, 255, cv2.THRESH_BINARY)
```

```
num_labels, labels = cv2.connectedComponents(binary)

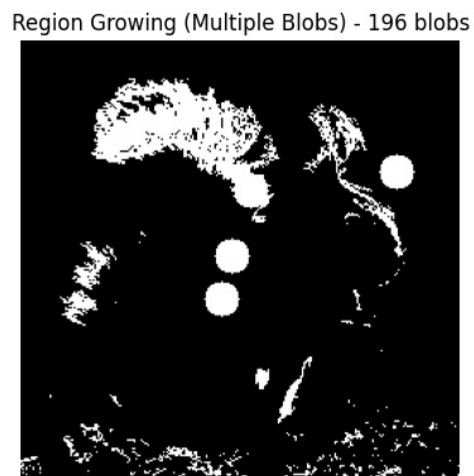
# Perform region growing from each seed
segmented_total = np.zeros_like(img)
for label in range(1, num_labels):
    ys, xs = np.where(labels == label)
    if len(xs) > 0:
        seed = (ys[len(ys)//2], xs[len(xs)//2])
        region = region_growing(img, seed, threshold=15)
        segmented_total = cv2.bitwise_or(segmented_total, region)

# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image with Blobs")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(segmented_total, cmap='gray')
plt.title(f"Region Growing (Multiple Blobs) - {num_labels-1} blobs")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT



Segmentation (Split and Merge)

AIM

Implement split and merge procedure for segmenting the image with quad-tree regions.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Function to check homogeneity
def is_homogeneous(region, threshold=10):
    """Check if a region is homogeneous based on pixel intensity
    variance."""
    return np.std(region) < threshold

# Recursive splitting function
def split(image, min_size, threshold):
    h, w = image.shape
    if h <= min_size or w <= min_size or is_homogeneous(image, threshold):
        mean_val = np.mean(image)
        return np.full_like(image, mean_val)
    else:
        mid_h, mid_w = h // 2, w // 2
        top_left = split(image[:mid_h, :mid_w], min_size, threshold)
        top_right = split(image[:mid_h, mid_w:], min_size, threshold)
        bottom_left = split(image[mid_h:, :mid_w], min_size, threshold)
        bottom_right = split(image[mid_h:, mid_w:], min_size, threshold)
        return np.vstack((np.hstack((top_left, top_right)),
                            np.hstack((bottom_left, bottom_right))))

# Merge operation (basic smoothing of split result)
def merge(image, merge_threshold=15):
    merged = cv2.medianBlur(image.astype(np.uint8), 3)
    return merged

# Load grayscale image
img = cv2.imread("image.png", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Try different minimum quad-tree dimensions
```

```
min_sizes = [64, 32, 16]

# Plot results
plt.figure(figsize=(15, 5))

plt.subplot(1, len(min_sizes) + 1, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image", fontsize=12)
plt.axis("off")

for i, min_size in enumerate(min_sizes, start=2):
    split_result = split(img, min_size=min_size, threshold=10)
    merged_result = merge(split_result, merge_threshold=15)
    plt.subplot(1, len(min_sizes) + 1, i)
    plt.imshow(merged_result, cmap='gray')
    plt.title(f"Split & Merge\n(min={min_size})", fontsize=11)
    plt.axis("off")

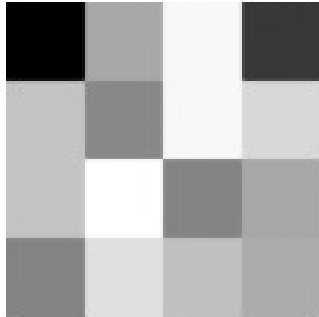
plt.tight_layout(pad=3.0)
plt.show()
```

OUTPUT

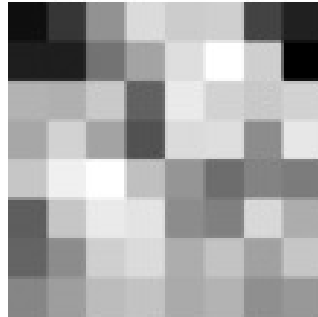
Original Image



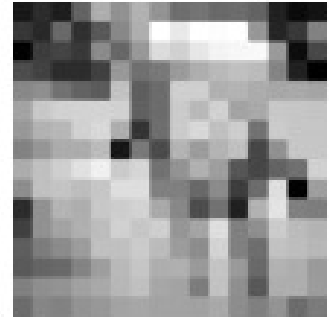
Split & Merge
(min=64)



Split & Merge
(min=32)



Split & Merge
(min=16)



Segmentation (Watershed)

AIM

Write a program to segment the blobs using watershed transform.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
img = cv2.imread("image.png")
img = cv2.resize(img, (256, 256))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Threshold to get binary blobs
# Converts to binary (foreground = white blobs)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Noise removal (Morphological operations)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background area
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Finding sure foreground area
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.5 * dist_transform.max(), 255,
    ↪ 0)

# Unknown region (background - foreground)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labelling
_, markers = cv2.connectedComponents(sure_fg)

# Add one to all labels (so background is not 0)
markers = markers + 1
```

```
# Mark unknown region with 0
markers[unknown == 255] = 0

# Apply Watershed transform
markers = cv2.watershed(img, markers)
segmented = img.copy()
segmented[markers == -1] = [0, 0, 255] # mark boundaries in red

# Display all results
plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(binary, cmap='gray')
plt.title("Binary Thresholded Blobs")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(dist_transform, cmap='gray')
plt.title("Distance Transform")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB))
plt.title("Watershed Segmentation")
plt.axis("off")

plt.tight_layout(pad=3)
plt.show()
```

OUTPUT

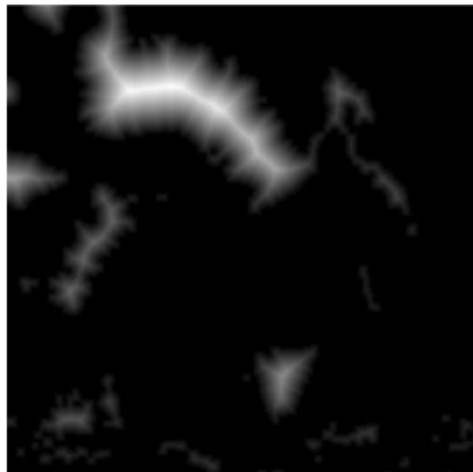
Original Image



Binary Thresholded Blobs



Distance Transform



Watershed Segmentation



Optical Flow (Farneback)

AIM

Compute dense optical flow between the frames using the Farneback algorithm and visualize the motion vectors as arrows over the first frame. Comment on differences in motion vectors in fast-moving vs slow-moving regions.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# Define the base directory and filenames as per instructions
image_dir = '/content/drive/MyDrive/image_2/'
file_name1 = '000000_10.png'
file_name2 = '000000_11.png'

# Construct full paths
frame1_path = os.path.join(image_dir, file_name1)
frame2_path = os.path.join(image_dir, file_name2)

# Load the two consecutive grayscale images
frame1 = cv2.imread(frame1_path, cv2.IMREAD_GRAYSCALE)
frame2 = cv2.imread(frame2_path, cv2.IMREAD_GRAYSCALE)

# Ensure images are loaded correctly before proceeding
if frame1 is None or frame2 is None:
    # Use the constructed paths in the error message for clarity
    raise FileNotFoundError(f"One or both images could not be loaded.
    Please check paths: {frame1_path}, {frame2_path}")

print(f"Loaded frame 1 shape: {frame1.shape}")
print(f"Loaded frame 2 shape: {frame2.shape}")

# Parameters: prev, next, flow, pyr_scale, levels, winsize, iterations,
poly_n, poly_sigma, flags
flow = cv2.calcOpticalFlowFarneback(frame1, frame2, None, 0.5, 3, 15, 3, 5,
↪ 1.2, 0)
```

```
print("Dense optical flow computed successfully.")
print(f"Flow shape: {flow.shape}")

def draw_flow(img, flow, step=16):
    h, w = img.shape[:2]
    y, x = np.mgrid[step/2:h:step, step/2:w:step].reshape(2, -1).astype(int)
    fx, fy = flow[y, x].T

    lines = np.vstack([x, y, x + fx, y + fy]).T.reshape(-1, 2, 2)
    lines = np.int32(lines + 0.5)
    vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

    # Scale factor for arrows, adjust as needed for better visualization
    max_flow_mag = np.max(np.sqrt(fx**2 + fy**2))
    scale_factor = 5.0 / (max_flow_mag + 1e-6) if max_flow_mag > 0 else 1.0

    for (x1, y1), (x2, y2) in lines:
        # Draw the main line of the arrow
        cv2.line(vis, (x1, y1), (int(x1 + fx[lines.tolist().index([[x1,
y1], [x2,y2]])]*scale_factor), int(y1 +
fy[lines.tolist().index([[x1, y1], [x2,y2]])]*scale_factor))), (0,
255, 0), 1)
        # Draw the arrowhead
        # calculate the angle of the arrow
        angle = np.arctan2(y1 - (y1 + fy[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor), x1 - (x1 + fx[lines.tolist().index([[x1,
y1], [x2,y2]])]*scale_factor))
        # Draw two short lines for the arrowhead
        cv2.line(vis, (int(x1 + fx[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor), int(y1 + fy[lines.tolist().index([[x1,
y1], [x2,y2]])]*scale_factor)),
                (int(x1 + fx[lines.tolist().index([[x1, y1], [
x2,y2]])]*scale_factor + 9 * np.cos(angle + np.pi / 4)),
                int(y1 + fy[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor + 9 * np.sin(angle + np.pi /
4))), (0, 255, 0), 1)
        cv2.line(vis, (int(x1 + fx[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor), int(y1 + fy[lines.tolist().index([[x1,
y1], [x2,y2]])]*scale_factor)),
                (int(x1 + fx[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor + 9 * np.cos(angle - np.pi / 4)),
```

```
int(y1 + fy[lines.tolist().index([[x1, y1],
[x2,y2]])]*scale_factor + 9 * np.sin(angle - np.pi /
4))), (0, 255, 0), 1)

return vis

# Visualize the motion vectors
flow_image = draw_flow(frame1, flow)

plt.figure(figsize=(15, 8))
plt.imshow(flow_image)
plt.title('Dense Optical Flow (Farneback) with Motion Vectors')
plt.axis('off')
plt.show()
```

OUTPUT

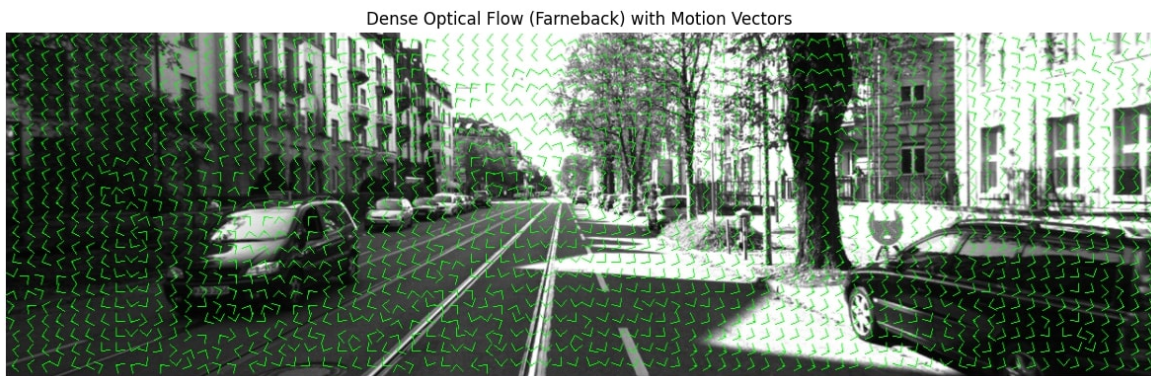
Loaded frame 1 shape: (375, 1242)

Loaded frame 2 shape: (375, 1242)

Dense optical flow computed successfully.

Flow shape: (375, 1242, 2)

Fast-moving regions show long, consistent motion vectors, while slow-moving or stationary areas exhibit short, scattered, or barely visible vectors.



Block Matching (SAD)

AIM

Select two consecutive frames from a sequence. Divide the frames into 16×16 pixel blocks. For each block in the first frame, find the best matching block in the next frame using SAD (Sum of Absolute Differences). Compute and display motion vectors as arrows over the first frame. Comment on:

- Which regions exhibit larger motion vectors and why?
- How stationary regions appear in terms of motion vectors?
- How the choice of block size affects the accuracy of motion estimation?

PROGRAM

```
def block_match(frame1, frame2, block_size, search_range):
    h1, w1 = frame1.shape
    h2, w2 = frame2.shape

    # Calculate number of blocks
    num_blocks_y = h1 // block_size
    num_blocks_x = w1 // block_size

    # Initialize motion vectors array (dx, dy)
    motion_vectors = np.zeros((num_blocks_y, num_blocks_x, 2),
                               dtype=np.float32)

    for i in range(num_blocks_y):
        for j in range(num_blocks_x):
            # Current block in frame1
            y = i * block_size
            x = j * block_size
            current_block = frame1[y : y + block_size, x : x + block_size]

            min_sad = float('inf')
            best_dx, best_dy = 0, 0

            # Search within search_range in frame2
            for dy_search in range(-search_range, search_range + 1):
                for dx_search in range(-search_range, search_range + 1):
                    # Candidate block's top-left corner in frame2
                    candidate_y = y + dy_search
```

```
        candidate_x = x + dx_search

        # Check bounds for candidate block in frame2
        if (candidate_y >= 0 and candidate_y + block_size <= h2
            ↪ and
                candidate_x >= 0 and candidate_x + block_size <=
                    ↪ w2):

            candidate_block = frame2[candidate_y : candidate_y
                + block_size, candidate_x : candidate_x +
                    block_size]

            # Calculate SAD
            sad = np.sum(np.abs(current_block -
                ↪ candidate_block))

            if sad < min_sad:
                min_sad = sad
                best_dx = dx_search
                best_dy = dy_search

        motion_vectors[i, j, 0] = best_dx
        motion_vectors[i, j, 1] = best_dy

    return motion_vectors

def draw_block_flow(original_frame, block_motion_vectors, block_size,
    scale_factor=5.0):
    vis = cv2.cvtColor(original_frame, cv2.COLOR_GRAY2BGR)
    h, w = original_frame.shape[:2]
    num_blocks_y, num_blocks_x, _ = block_motion_vectors.shape

    for i in range(num_blocks_y):
        for j in range(num_blocks_x):
            # Center coordinates of the current block in the original frame
            center_x = int(j * block_size + block_size / 2)
            center_y = int(i * block_size + block_size / 2)

            # Motion vector for this block
            fx, fy = block_motion_vectors[i, j]
```

```
# Scale the motion vectors for visualization
fx_scaled = int(center_x + fx * scale_factor)
fy_scaled = int(center_y + fy * scale_factor)

# Draw the arrow
cv2.arrowedLine(vis, (center_x, center_y), (fx_scaled,
fy_scaled), (0, 255, 0), 1, tipLength=0.2)

return vis

block_size = 16
search_range = 15

# Compute motion vectors using block_match
block_vectors = block_match(frame1, frame2, block_size, search_range)
print(f"Computed block motion vectors with shape: {block_vectors.shape}")

# Visualize the motion vectors
block_flow_image = draw_block_flow(frame1, block_vectors, block_size)
print("Generated block flow visualization image.")

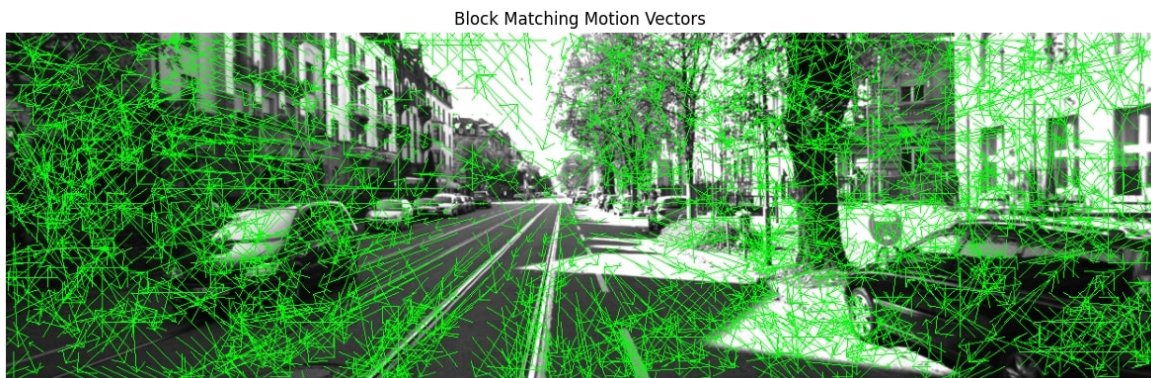
# Display the block_flow_image
plt.figure(figsize=(15, 8))
plt.imshow(block_flow_image)
plt.title('Block Matching Motion Vectors')
plt.axis('off')
plt.show()
```

OUTPUT

Computed block motion vectors with shape: (23, 77, 2)

Generated block flow visualization image.

Larger motion vectors are found in areas with significant movement (e.g., foreground objects), or due to camera motion. Stationary regions show small, sometimes inconsistent vectors due to noise or ego-motion. Smaller block sizes capture finer details but are sensitive to noise, whereas larger blocks are more robust but less precise.



Frame Prediction

AIM

Select two consecutive frames from a sequence. Using motion vectors (obtained from block matching), predict the next frame from the first frame. Compute the residual difference image between the predicted frame and the actual next frame. Visualize the original frame, predicted frame, and residual image side by side. Comment on:

- How accurately does the predicted frame match the actual frame?
- Which regions have larger residual errors and why?
- How motion vectors contribute to reducing frame-to-frame redundancy?

PROGRAM

```
def predict_frame(original_frame, block_motion_vectors, block_size):
    h,w = original_frame.shape
    predicted_frame = np.zeros_like(original_frame, dtype=np.uint8)

    num_blocks_y, num_blocks_x, _ = block_motion_vectors.shape

    for i in range(num_blocks_y):
        for j in range(num_blocks_x):
            # Top-left corner of the current block in the original frame
            y_orig = i * block_size
            x_orig = j * block_size

            # Motion vector for this block (dx, dy)
            fx, fy = block_motion_vectors[i, j]

            # Top-left corner of the block's new position in the predicted
            ↪ frame
            y_pred = int(y_orig + fy)
            x_pred = int(x_orig + fx)

            # Extract the block from the original frame
            current_block = original_frame[y_orig : y_orig + block_size,
            x_orig : x_orig + block_size]

            # Place the block in the predicted frame, handling boundaries
            target_y_start = max(0, y_pred)
            target_x_start = max(0, x_pred)
```



```
target_y_end = min(h, y_pred + block_size)
target_x_end = min(w, x_pred + block_size)

# Determine the corresponding slice of the block to be placed
source_y_start = 0
source_x_start = 0
if y_pred < 0:
    source_y_start = -y_pred
if x_pred < 0:
    source_x_start = -x_pred

source_y_end = block_size - (y_pred + block_size - target_y_end)
source_x_end = block_size - (x_pred + block_size - target_x_end)

if source_y_start < source_y_end and source_x_start <
    ↪ source_x_end:
    predicted_frame[target_y_start:target_y_end,
        target_x_start:target_x_end] = \
        current_block[source_y_start:source_y_end,
            source_x_start:source_x_end]

return predicted_frame
predicted_frame = predict_frame(frame1, block_vectors, block_size)
# Compute residual difference image
residual_image = cv2.absdiff(frame2, predicted_frame)
print("Predicted frame and residual image computed.")
# Display frame1, predicted_frame, and residual_image side by side
plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(frame1, cmap='gray')
plt.title('Original Frame (Frame 1)')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(predicted_frame, cmap='gray')
plt.title('Predicted Frame')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(residual_image, cmap='gray')
plt.title('Residual Image (Actual Frame 2 - Predicted Frame)')
plt.axis('off')
plt.show()
```

OUTPUT

The predicted frame matches well in areas of simple motion but shows errors at object boundaries, occlusions, and complex movements. Larger residual errors indicate these problematic regions. Motion vectors reduce redundancy by enabling efficient differential encoding for video compression.

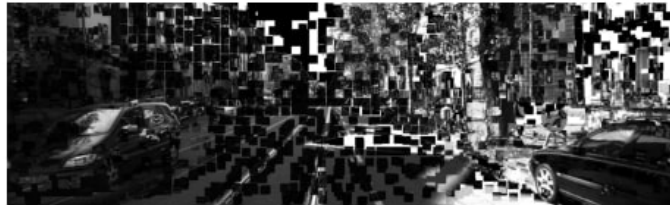
Original Frame (Frame 1)



Predicted Frame



Residual Image (Actual Frame 2 - Predicted Frame)



Stereo Disparity & Depth Map

AIM

Select a stereo image pair (left and right images) from a KITTI sequence. Compute the disparity map using StereoSGBM. Convert the disparity map to a depth map using the camera focal length and baseline. Normalize and visualize both the disparity map and the depth map. Comment on:

- How disparity values relate to object distance?
- Which regions appear closer or farther in the depth map?

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

# Define camera focal length and baseline (typical KITTI values)
focal_length = 721.5377 # pixels
baseline = 0.5379 # meters

# Convert the disparity map to actual disparity values by dividing by 16.0
disparity_actual = disparity.astype(np.float32) / 16.0

# Compute the depth map using the formula: depth = (focal_length *
#                                     baseline) / (disparity_actual + epsilon)
epsilon = 1e-6 # Small value to avoid division by zero
depth_map = (focal_length * baseline) / (disparity_actual + epsilon)

# Replace any infinite or very large depth values with a maximum valid
# depth or set them to zero.
# Set depth where disparity_actual is <= 0 (or original disparity was -16)
# to 0 or a very large number (NaN)
max_depth_val = 100.0 # meters, a reasonable upper bound for visualization
# → clarity
depth_map[disparity_actual <= 0] = max_depth_val
depth_map[depth_map > max_depth_val] = max_depth_val # Cap depth at
# → max_depth_val
depth_map[depth_map < 0] = 0

# Normalize the actual disparity map to an 8-bit image for visualization
```

```
disparity_for_viz = cv2.normalize(disparity_actual, None, 255, 0,
                                  cv2.NORM_MINMAX, cv2.CV_8U)

# Normalize the computed depth map to an 8-bit image for visualization
depth_for_viz = cv2.normalize(depth_map, None, 255, 0, cv2.NORM_MINMAX,
                               ↪ cv2.CV_8U)

print("Depth map computed and normalized successfully.")

# Display the normalized disparity map and the normalized depth map side by
↪ side
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
plt.imshow(disparity_for_viz, cmap='viridis')
plt.title('Normalized Actual Disparity Map')
plt.colorbar(label='Disparity Value (Normalized)')
plt.axis('off')

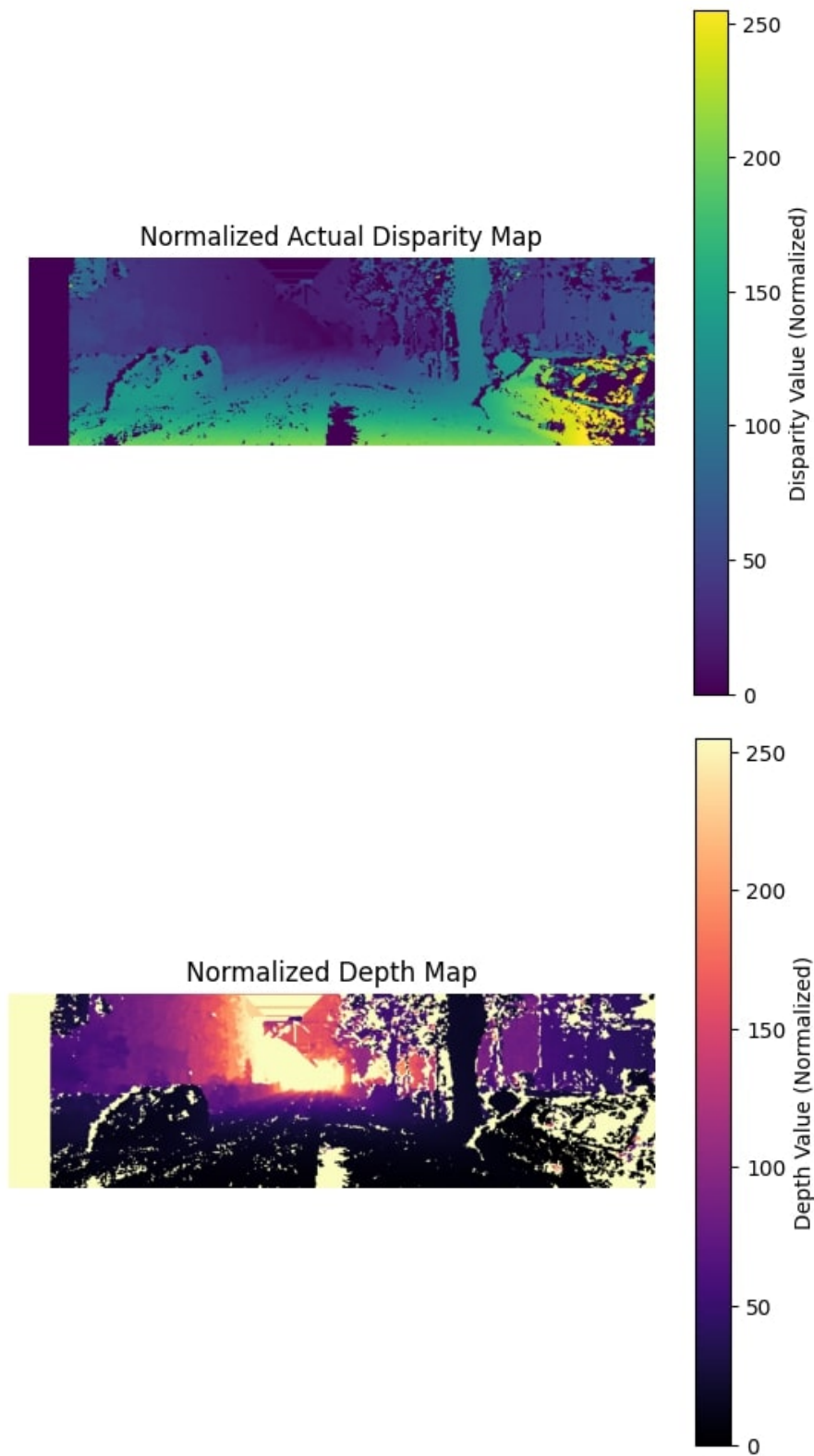
plt.subplot(1, 2, 2)
plt.imshow(depth_for_viz, cmap='magma')
plt.title('Normalized Depth Map')
plt.colorbar(label='Depth Value (Normalized)')
plt.axis('off')

plt.show()
```

OUTPUT

Depth map computed and normalized successfully.

Disparity values are inversely related to object distance: higher disparity indicates closer objects. In the depth map, closer regions appear darker, and farther regions appear brighter (with the 'magma' colormap).



Obstacle Detection

AIM

Using a stereo image pair, compute the disparity map and convert it to a depth map. Detect and highlight objects that are closer than a specified depth threshold. Visualize the result by overlaying detected obstacles on the original image or depth map. Comment on:

- Which objects are detected as obstacles and why?
- How changing the depth threshold affects detection results?

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Define a depth_threshold value in meters
depth_threshold = 20.0 # meters. Adjustable

print(f"Using depth threshold: {depth_threshold} meters")

# Create a binary mask (obstacles_mask)
# Pixels with depth values less than depth_threshold are marked as
#   obstacles (True or 255).
# Also, filter out regions where disparity was invalid (depth_map was
#   max_depth_val or 0 initially set by handling).
# We want to detect objects that are closer than the threshold, not just
#   undefined regions.
# Assume max_depth_val (100.0) represents 'too far/invalid', so we only
#   consider valid depth values < max_depth_val.
obstacles_mask = (depth_map < depth_threshold) & (depth_map > 0) # Ensure
#   depth is positive and less than threshold

# Convert mask to uint8 for visualization (255 for obstacles, 0 for non-
#   obstacles)
obstacles_mask_uint8 = (obstacles_mask * 255).astype(np.uint8)

# Convert the original frame1 (grayscale) to a color image
frame1_color = cv2.cvtColor(frame1, cv2.COLOR_GRAY2BGR)

# Overlay the obstacles_mask onto the colorized frame1
```

```
highlighted_obstacles_image = frame1_color.copy()
# Find the coordinates where obstacles are detected
y_coords, x_coords = np.where(obstacles_mask)

# Color these regions in red (BGR format for OpenCV) on the original image
highlighted_obstacles_image[y_coords, x_coords] = [0, 0, 255] # Red color
[B, G, R]

print("Obstacles detected and highlighted.")

plt.figure(figsize=(25, 8))

plt.subplot(1, 3, 1)
plt.imshow(frame1, cmap='gray')
plt.title('Original Left Frame')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(depth_for_viz, cmap='magma') # Using the normalized depth map
    for visualization
plt.title('Normalized Depth Map')
plt.colorbar(label='Depth Value (Normalized)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(highlighted_obstacles_image, cv2.COLOR_BGR2RGB))
plt.title(f'Objects Closer Than {depth_threshold}m (Highlighted Red)')
plt.axis('off')

plt.show()
```

OUTPUT

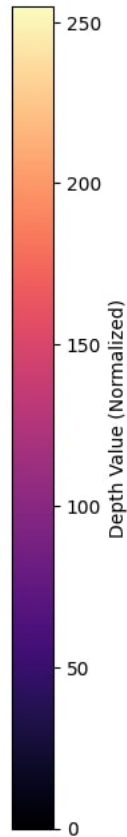
Using depth threshold: 20.0 meters
Obstacles detected and highlighted.

Objects like the immediate road and nearby vehicles are detected as obstacles because their depth is below the set threshold. Decreasing the threshold identifies only very close objects, while increasing it detects more distant ones but risks more false positives.

Original Left Frame



Normalized Depth Map



Objects Closer Than 20.0m (Highlighted Red)

